0. **Code:** Final Uppal Harsh.zip

1. FinalTest / A. LargestBaseN / A. LargestBaseN / Program.cs
2. FinalTest / B. InsertionCounts / InsertionCounts / Program.cs
3. FinalTest / C. Dictionary / C. Dictionary / Program.cs

1. **FinalTest / A. LargestBaseN / A. LargestBaseN / Program.cs**

```
using System.Buffers.Text;

namespace A._LargestBaseN
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Enter an integer");
            int num = int.Parse(Console.ReadLine());
            int sum = 0;
            int base_n = 0;
            while (true)
            {
                Console.WriteLine("Enter Base-N value");
                base_n = int.Parse(Console.ReadLine());
                if (base_n > 0)
                {
                    break;
                }
            }
            //If number is less than 1 then enter the value again
              (positive value)
            if (num < 0)
            {
                num = -num;
            }
            int n = 0;
            int largest = int.MinValue;
            while (true)
            {
```

A1

A2

A3

A5

It's okay here

1. **FinalTest / A. LargestBaseN / A. LargestBaseN / Program.cs**

```
                if (num == base_n)
                {
                    n = num / base_n;
                    if (n > largest)
                    {
                        largest = n;
                    }
                    sum += n;
                    break;
                }
                else
                {
                    n = num % base_n;
                    if (n > largest)
                    {
                        largest = n;
                    }
                    sum += n;
                    num = num / base_n;
                    if (num == 0)
                    {
                        break;
                    }
                }
            }
            Console.WriteLine("Sum is: " + sum);
            Console.WriteLine("Largest Base-N digit is: " + largest);
        }
    }
}
```

1

Does not need a special case.

A5

n = 4/4 = 1

Same code

A3

A5

A4

A3

A3

10

2. FinalTest / B. InsertionCounts / InsertionCounts / Program.cs

```
namespace InsertionCounts
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int[] array = {
                23,
                1,
                10,
                5,
                2
            };
            int[] array1 = {
                2,
                8,
                7,
                6,
                9
            };
            int[] array2 = {
                9,
                8,
                2,
                7,
                6,
                1
            };
            Console.WriteLine(String.Join(", ", InsertionCounts(array)));
        }
        static int[] InsertionCounts(int[] array)
```

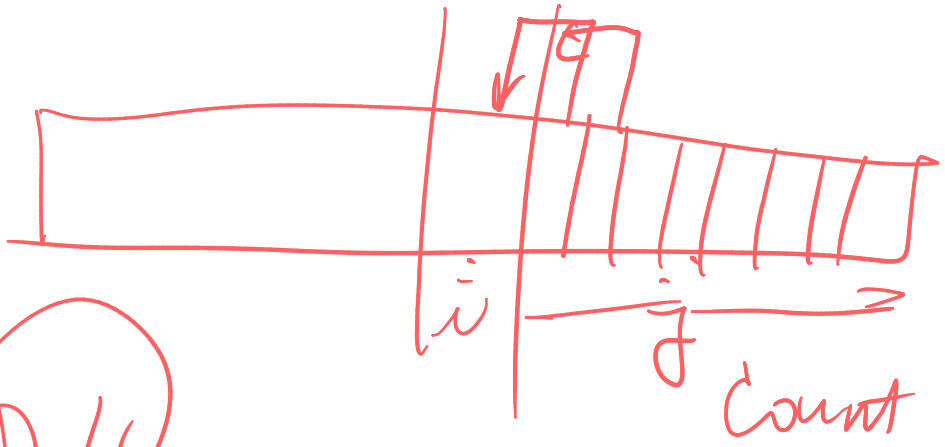
2. FinalTest / B. InsertionCounts / InsertionCounts / Program.cs

```
        {
            int[] result = new int[array.Length];
            if (array == null || array.Length == 0)
            {
                return result;
            }
            result[0] = 0;
            for (int i = 0; i < array.Length - 1; i++)
            {
                for (int j = i + 1; j < array.Length; j++)
                {
                    if (array[j] < array[i])
                    {
                        result[j]++;
                    }
                }
            }
            return result;
        }
    }
}
```

B2

OK

B3



B4

No Sorting.

3. FinalTest / C. Dictionary / C. Dictionary / Program.cs

```
namespace C._Dictionary
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int[] array = {
                1,
                2,
                3,
                2,
                7,
                3,
                4,
                4,
                5,
                5,
                4,
                5,
                1,
                6,
                1,
                1
            };
            Console.WriteLine(LargestNumSum(array));
        }
        static int LargestNumSum(int[] array)
        {
            if (array == null || array.Length == 0)
            {
                return 0;
```

C1

```
            }
            //Store array values in occurence dictionary
            var occurenceDict = new Dictionary < int,
                int > ();
            foreach(int element in array)
            {
                if (!occurenceDict.ContainsKey(element))
                {
                    occurenceDict.Add(element, 1);
                }
                else
                {
                    occurenceDict[element]++;
                }
            }
            //Create a list that contains unique count values from
              occurence dictionary and ]
            //copy this list's element to the maxDict as keys and make
              their respective value as 0 for now.
            var list = (occurenceDict.Values.Distinct());
            var maxDict = new Dictionary < int,
                int > ();
            foreach(int item in list)
            {
                maxDict.Add(item, 0);
            }
            foreach(var key in occurenceDict.Keys)
            {
                int freq = occurenceDict[key];
                if (maxDict.ContainsKey(freq))
                { .
```

C2

no!

not needed

C3

C3

3. **FinalTest / C. Dictionary / C. Dictionary / Program.cs**

```
                    if (key > maxDict[freq])
                    {
                        maxDict[freq] = key;
                    }
                }
            }
            int sum = 0;
            foreach(var item in maxDict)
            {
                int product = item.Key * item.Value;
                sum += product;
            }
            return sum;

        }
    }
}
```

C3

C4

C6

Used Distinct, Could use .ContainsKey( ) || key > maxDict[freq]

! maxFreq.